Atelier 7 : GitLab CI/CD + SonarQube Cloud

Niveau débutant · HTML · CSS · JavaScript · GitLab CI/CD · SonarQube Cloud · GitLab Pages

L'atelier repose sur deux versions du même mini-projet :

- V1 : code correct, analyse SonarQube et déploiement.
- V2 : code volontairement incorrect, nouvelle analyse SonarQube, consultation du dashboard et observation du comportement du pipeline.

Pour la V1 Nous conservons uniquement deux jobs : `sonarqube-check` et `deploy-pages`. Aucun job `validate`, aucun test automatisé.

Partie 1 — Présentation de SonarQube Cloud

1. Qu’est-ce que SonarQube Cloud ?



SonarQube Cloud est une plateforme d’analyse automatique de la qualité et de la sécurité du code source.

Elle s’intègre à GitLab CI/CD pour analyser le code, identifier des problèmes potentiels et évaluer si les critères de qualité définis pour le projet sont respectés.

SonarQube aide les développeurs à détecter des problèmes avant le déploiement. Il ne remplace ni les tests, ni la revue de code, ni le jugement du développeur.

Comment fonctionne-t-il ?

1. Le développeur pousse son code sur GitLab.
2. GitLab déclenche le pipeline CI/CD.
3. Le scanner analyse le code et envoie les résultats à SonarQube Cloud.
4. SonarQube calcule les métriques et évalue le Quality Gate.
5. GitLab utilise le résultat du job pour autoriser ou bloquer le déploiement.

2. Les principales métriques de SonarQube

B **Bugs — Fiabilité**

Défauts de programmation pouvant provoquer un comportement incorrect.

Exemple : additionner le prix et la quantité au lieu de les multiplier.

V **Vulnerabilities — Sécurité**

Failles de sécurité pouvant être exploitées.

Exemple : une injection XSS possible à cause de données non fiables insérées dans `innerHTML`.

S **Security Hotspots — Revue de sécurité**

Parties du code qui nécessitent une vérification humaine.

Exemple : une communication HTTP non chiffrée à examiner selon son contexte.

Sm

Code Smells — Maintenabilité

Problèmes qui rendent le code plus difficile à comprendre ou à modifier.

Exemple : une fonction trop complexe ou une logique inutilement répétée.

C

Coverage — Couverture des tests

Pourcentage du code exécuté par les tests automatisés.

Cette métrique nécessite un rapport de couverture. Elle ne sera pas exploitée dans notre atelier simplifié sans tests.

D

Duplications — Code dupliqué

Pourcentage de code identifié comme répété.

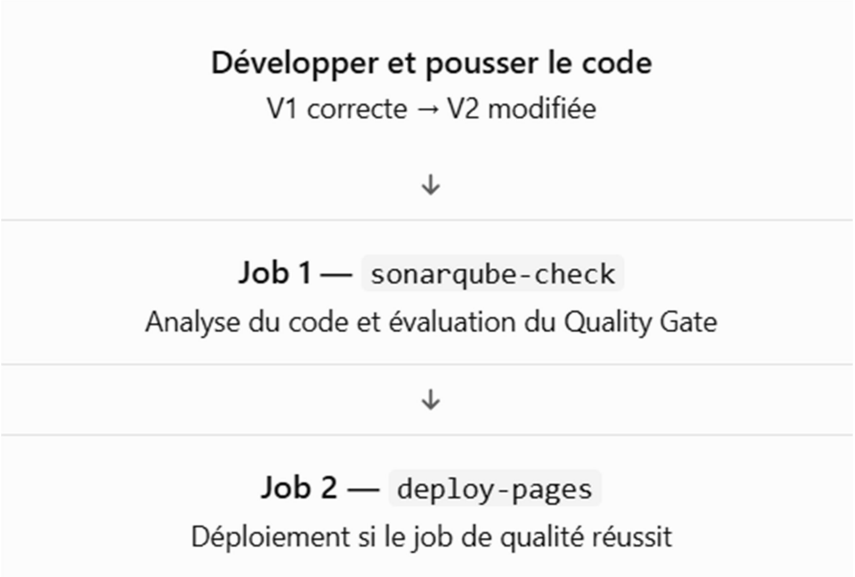
Exemple : copier plusieurs fois le même bloc de calcul au lieu de le réutiliser.

3. Qu’est-ce que le Quality Gate ?

Le Quality Gate est un ensemble de conditions permettant de déterminer si le code satisfait les exigences de qualité du projet.

Partie 2 — Schéma de l'atelier

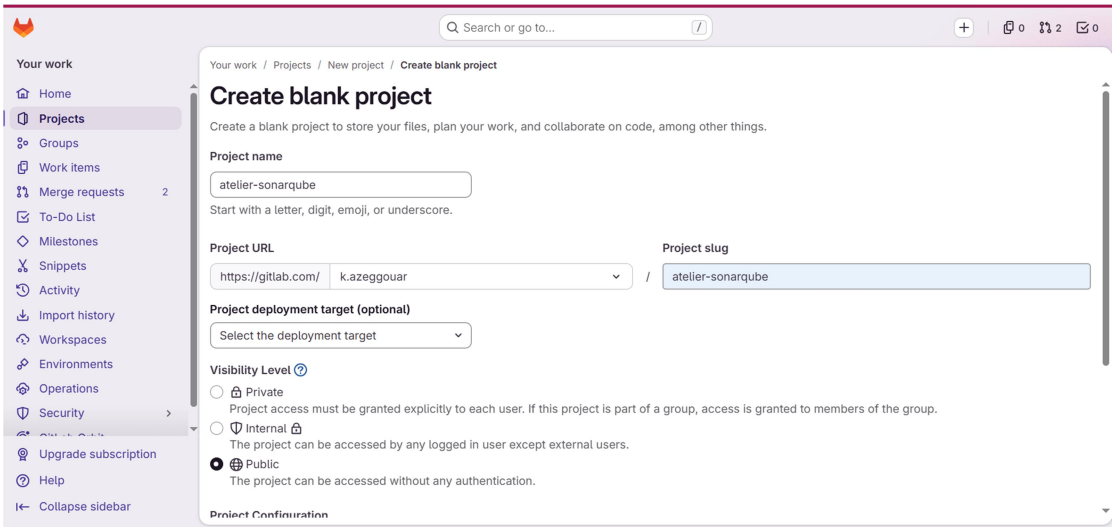
Nous allons réaliser un mini-site web avec deux versions : une V1 correcte, puis une V2 volontairement incorrecte.



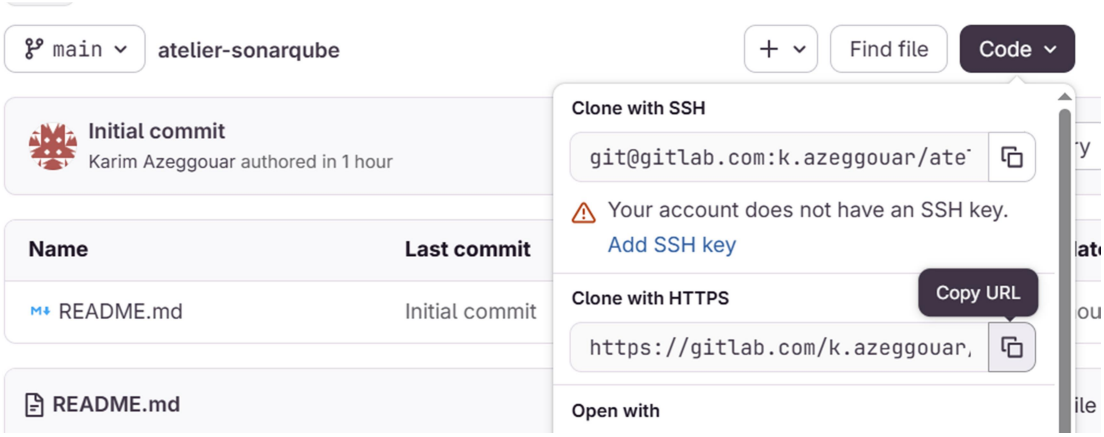
Le but est de voir les résultats de l'analyse SonarQube après le push de la V2. Nous n'ajoutons aucun job de validation séparé.

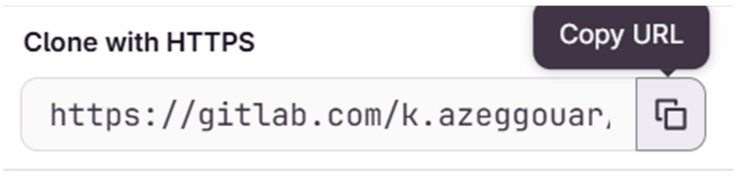
Partie 3 — Réalisation de l'atelier

Étape 1 — Créer le projet

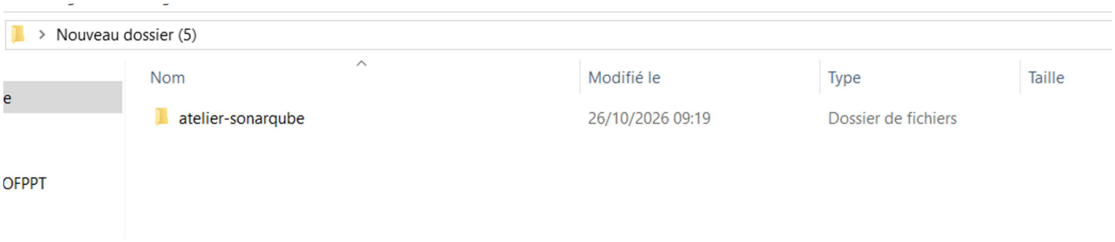


2-cloner le repo en local :

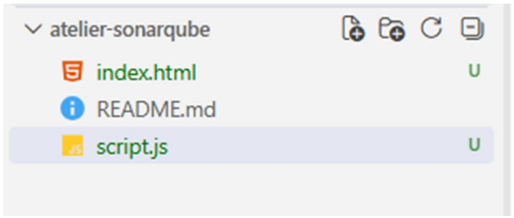





```
C:\Users\Admin\Desktop\Nouveau dossier (5)>git clone https://gitlab.com/k.azeggouar/atelier-sonarqube.git
```



3-Créer l'application



Créer la page index.html

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Atelier CI/CD</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Atelier GitLab CI/CD</h1>
  <p id="message">Version 1 : code correct</p>
  <button id="btn">Cliquer ici</button>

  <script src="script.js"></script>
</body>
</html>
```

Créer script.js — V1 correcte

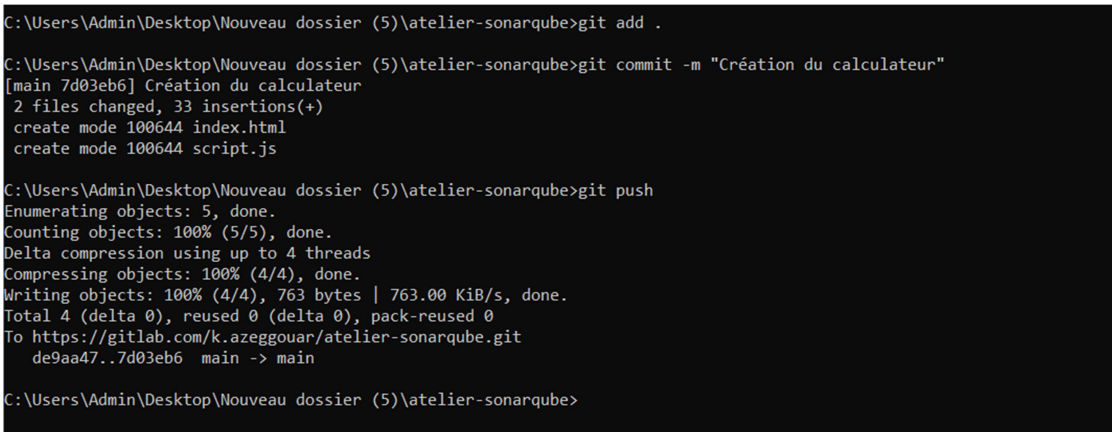
```
const bouton = document.getElementById("btn");
const message = document.getElementById("message");

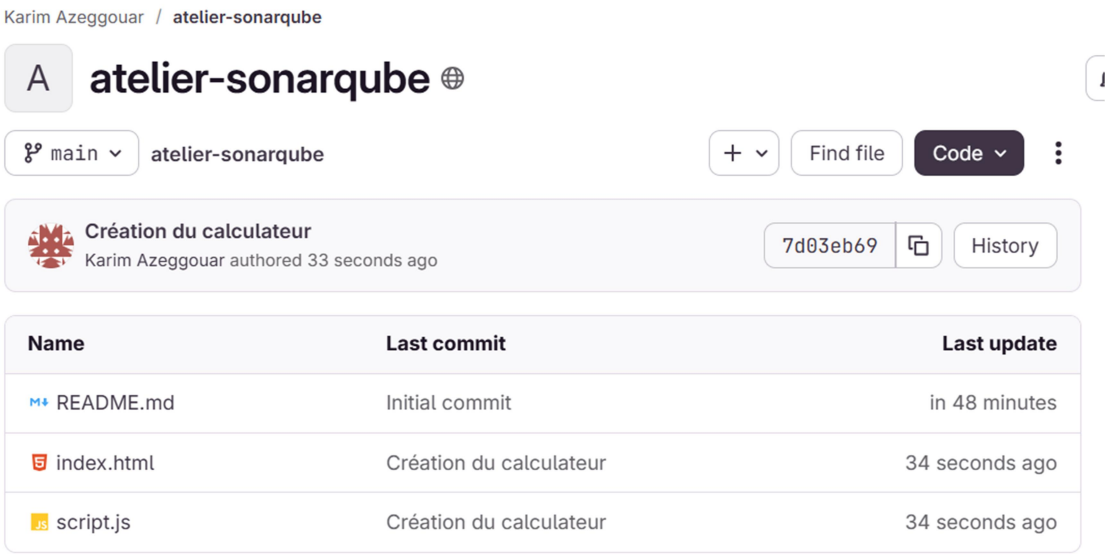
bouton.addEventListener("click", function () {
  message.textContent = "Le JavaScript fonctionne correctement !";
});
```

Étape 2 — Envoyer le projet sur GitLab

1. Crée un projet vierge nommé `atelier-sonarqube` sur GitLab.
2. Ouvre un terminal dans le dossier du projet.
3. Exécute les commandes suivantes :

```
git add .
git commit -m "Création du calculateur"
git push
```

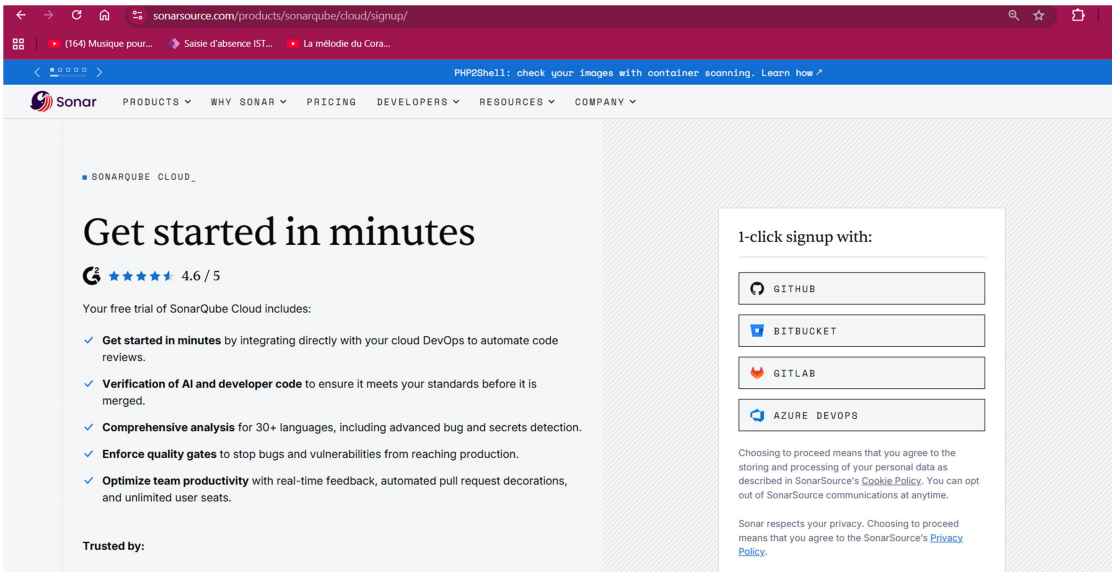


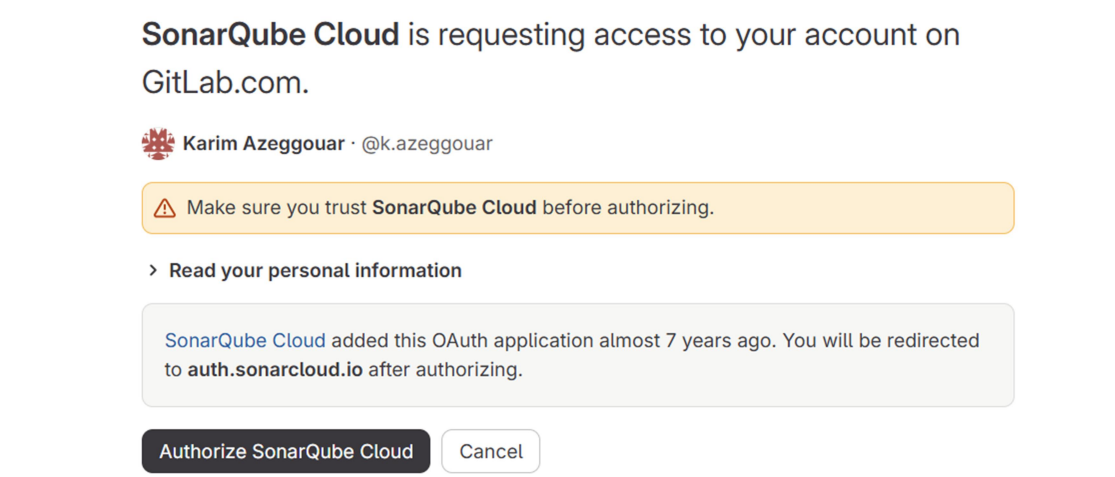


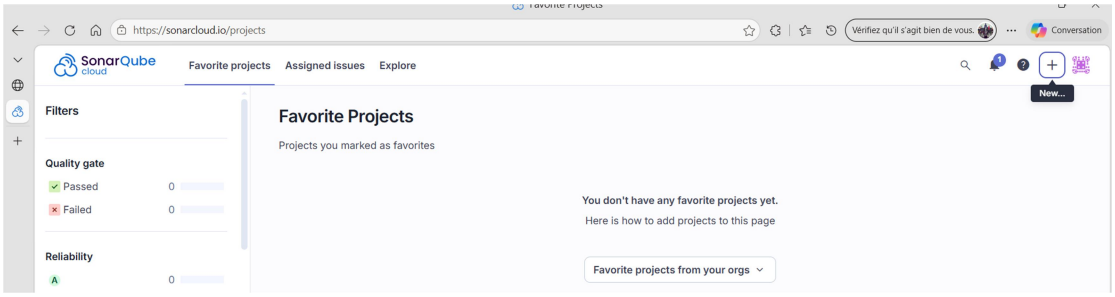
Étape 3 — Préparer SonarQube Cloud

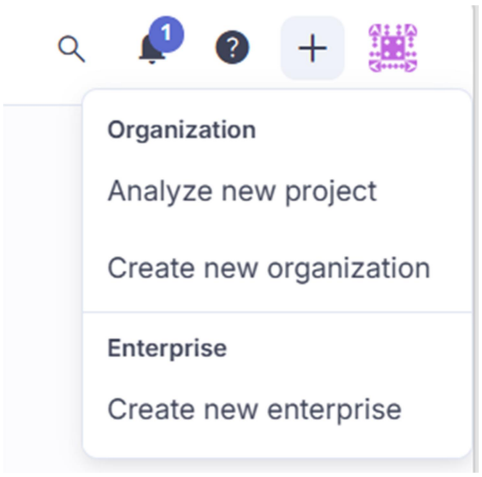
- Ouvre SonarQube Cloud. Et se connecter avec GitLab

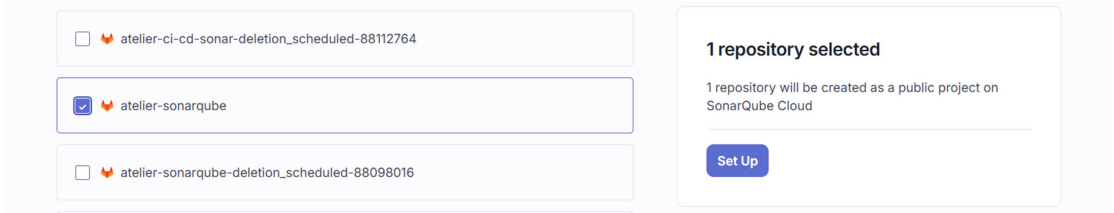


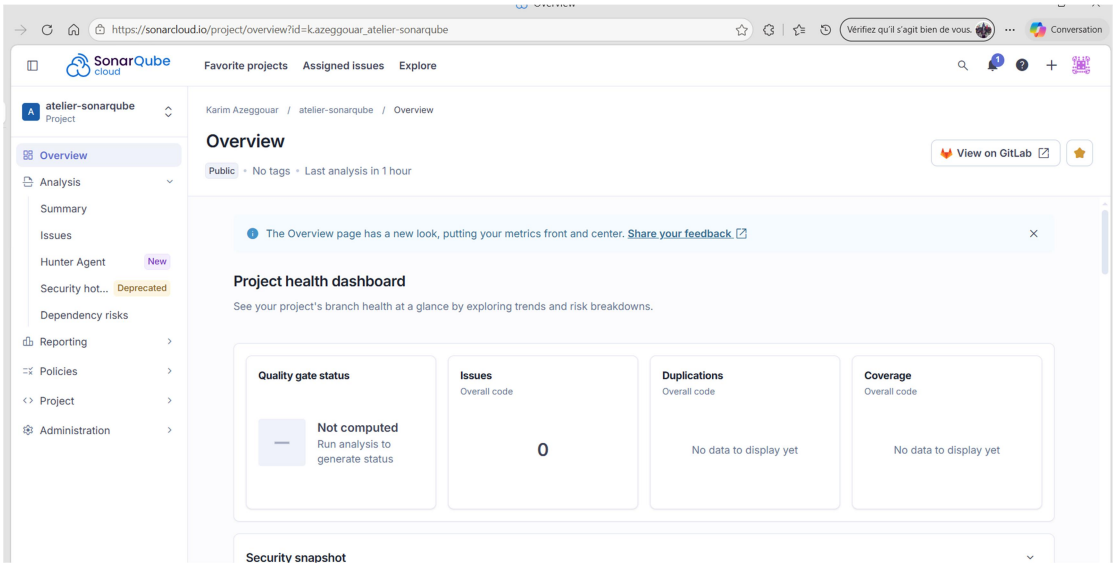

sonarsource.com/products/sonarqube/cloud/signup/
(164) Musique pour...
Saisie d'absence IST...
La mélodie du Cora...
PHP2Shell: check your images with container scanning. Learn how
Sonar
PRODUCTS
WHY SONAR
PRICING
DEVELOPERS
RESOURCES
COMPANY
SONARQUBE CLOUD_
Get started in minutes
4.6 / 5
Your free trial of SonarQube Cloud includes:
Get started in minutes by integrating directly with your cloud DevOps to automate code reviews.
Verification of AI and developer code to ensure it meets your standards before it is merged.
Comprehensive analysis for 30+ languages, including advanced bug and secrets detection.
Enforce quality gates to stop bugs and vulnerabilities from reaching production.
Optimize team productivity with real-time feedback, automated pull request decorations, and unlimited user seats.
Trusted by:
1-click signup with:
GITHUB
BITBUCKET
GITLAB
AZURE DEVOPS
Choosing to proceed means that you agree to the storing and processing of your personal data as described in SonarSource's Cookie Policy. You can opt out of SonarSource communications at anytime.
Sonar respects your privacy. Choosing to proceed means that you agree to the SonarSource's Privacy Policy.

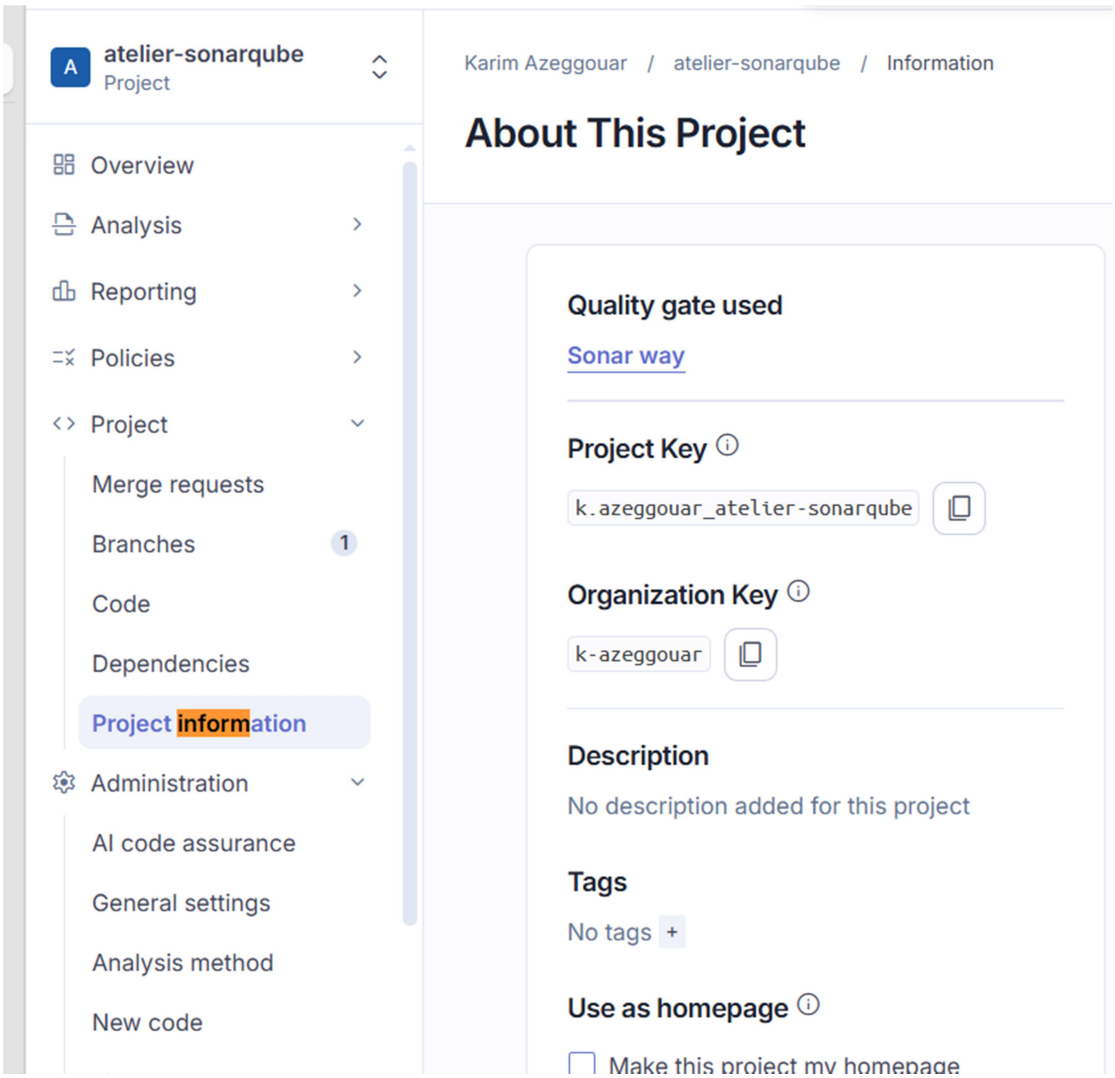

SonarQube Cloud is requesting access to your account on GitLab.com.
Karim Azeggouar · @k.azeggouar
Make sure you trust SonarQube Cloud before authorizing.
Read your personal information
SonarQube Cloud added this OAuth application almost 7 years ago. You will be redirected to auth.sonarcloud.io after authorizing.
Authorize SonarQube Cloud
Cancel


https://sonarcloud.io/projects
Vérifiez qu'il s'agit bien de vous.
Conversation
SonarQube cloud
Favorite projects
Assigned issues
Explore
New...
Filters
Quality gate
Passed 0
Failed 0
Reliability
A 0
Favorite Projects
Projects you marked as favorites
You don't have any favorite projects yet.
Here is how to add projects to this page
Favorite projects from your orgs


1
Organization
Analyze new project
Create new organization
Enterprise
Create new enterprise


atelier-ci-cd-sonar-deletion_scheduled-88112764
atelier-sonarqube
atelier-sonarqube-deletion_scheduled-88098016
1 repository selected
1 repository will be created as a public project on SonarQube Cloud
Set Up


https://sonarcloud.io/project/overview?id=k.azeggouar_atelier-sonarqube
Vérifiez qu'il s'agit bien de vous
Conversation
SonarQube cloud
Favorite projects
Assigned issues
Explore
atelier-sonarqube
Project
Overview
Analysis
Summary
Issues
Hunter Agent
New
Security hot...
Deprecated
Dependency risks
Reporting
Policies
Project
Administration
Karim Azeggouar / atelier-sonarqube / Overview
Overview
Public
No tags
Last analysis in 1 hour
View on GitLab
The Overview page has a new look, putting your metrics front and center. Share your feedback
Project health dashboard
See your project's branch health at a glance by exploring trends and risk breakdowns.
Quality gate status
Not computed
Run analysis to generate status
Issues
Overall code
0
Duplications
Overall code
No data to display yet
Coverage
Overall code
No data to display yet
Security snapshot



— Configurer SonarQube Cloud

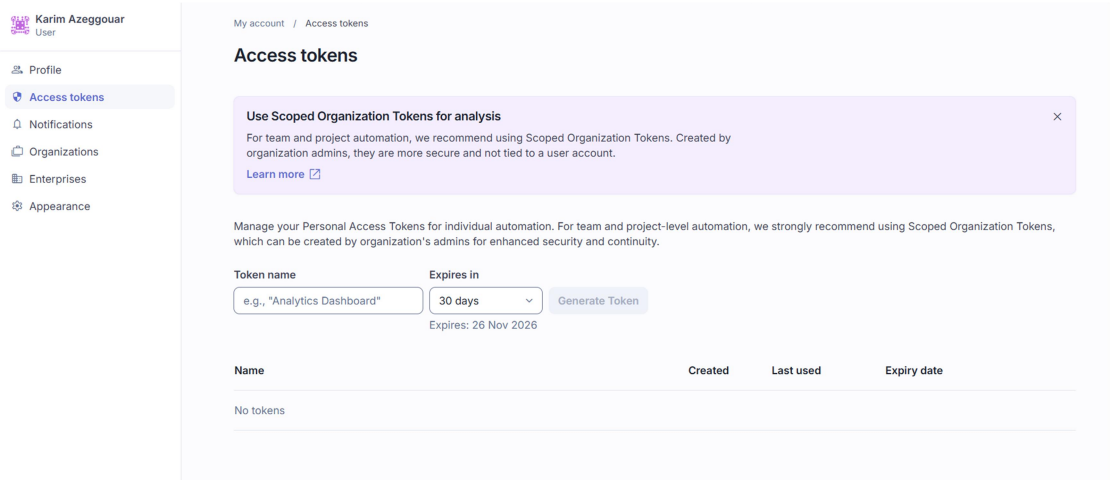
Créez le fichier `sonar-project.properties` :

```
sonar.projectKey=k.azeggouar_atelier-sonarqube
sonar.organization=k-azeggouar
sonar.projectName=Atelier CI CD SonarCloud

sonar.sources=.
sonar.sourceEncoding=UTF-8
sonar.exclusions=public/**

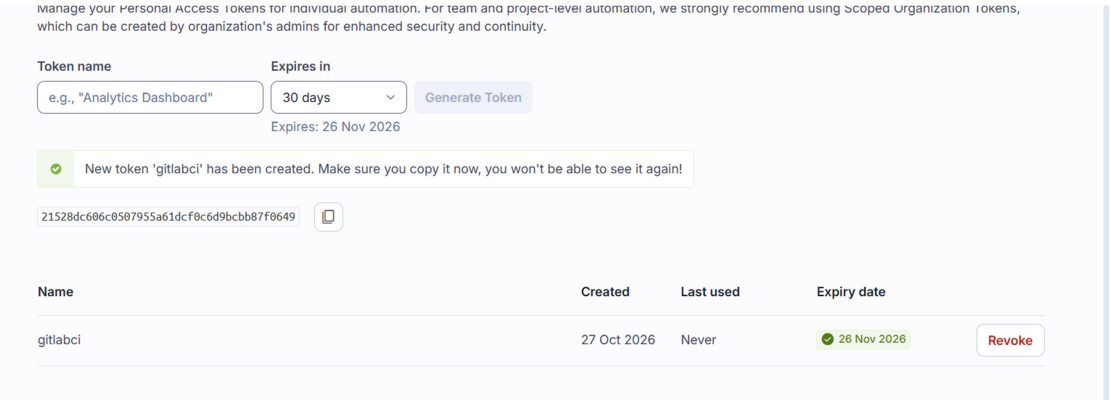
sonar.qualitygate.wait=true
```

-Créer le token pour GitLab CI/CD



Étape A — Dans SonarQube Cloud

1. Clique sur ton avatar.
2. Ouvre **My Account**, puis Access tockens
3. Génère un token avec un nom, par exemple `gitlab-ci`.
4. Copie-le immédiatement et conserve-le en sécurité.



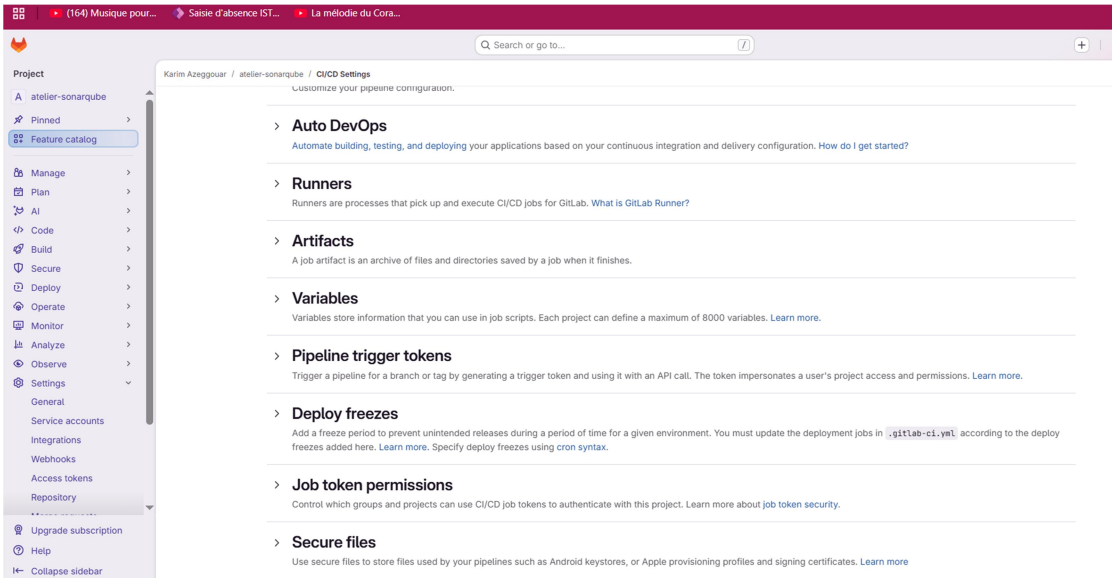


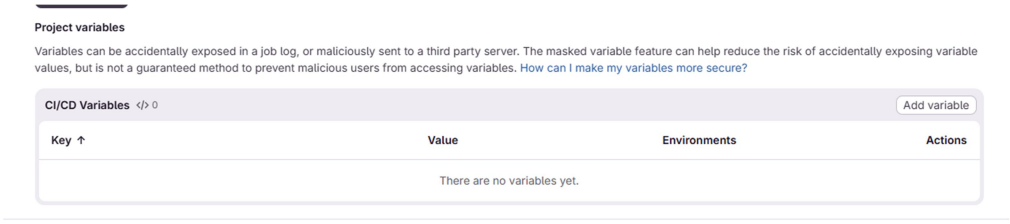
Étape B — Dans GitLab

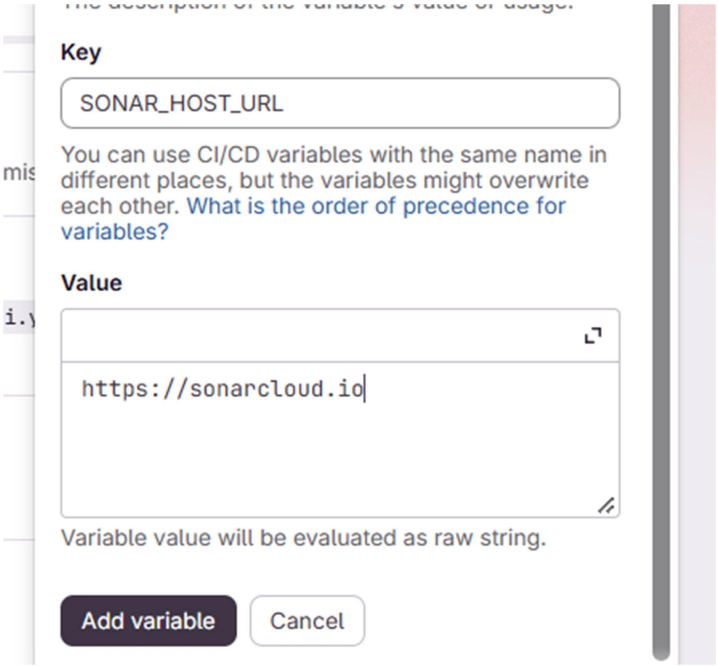
1. Ouvre ton dépôt GitLab.
2. Va dans **Settings** → **CI/CD** → **Variables**.
3. Ajoute une variable `SONAR_TOKEN` contenant le token généré.
4. Enregistre.

Dans GitLab, ouvrez **Settings → CI/CD → Variables** et vérifiez :

Variable	Valeur
`SONAR_HOST_URL`	`https://sonarcloud.io`
`SONAR_TOKEN`	Votre jeton SonarQube Cloud







Project variables

Variables can be accidentally exposed in a job log, or maliciously sent to a third party server. The masked variable feature can help reduce the risk of accidentally exposing variable values, but is not a guaranteed method to prevent malicious users from accessing variables. How can I make my variables more secure?

CI/CD Variables </> 2 — Reveal values — Add variable

Key ↑	Value	Environments	Actions
SONAR_HOST_URL Protected Masked	•••••	All (default)	
SONAR_TOKEN Protected Masked	•••••	All (default)	

— Configurer les deux jobs du pipeline

Créez `.gitlab-ci.yml` :

```yaml
stages:
  - quality
  - deploy

variables:
  GIT_DEPTH: "0"
  SONAR_USER_HOME: "${CI_PROJECT_DIR}/.sonar"

sonarqube-check:
  stage: quality
  image:
    name: sonarsource/sonar-scanner-cli:latest
    entrypoint: [""]
  script:
    - sonar-scanner
      -Dsonar.host.url="$SONAR_HOST_URL"
      -Dsonar.token="$SONAR_TOKEN"
  rules:
    - if: '$CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH'

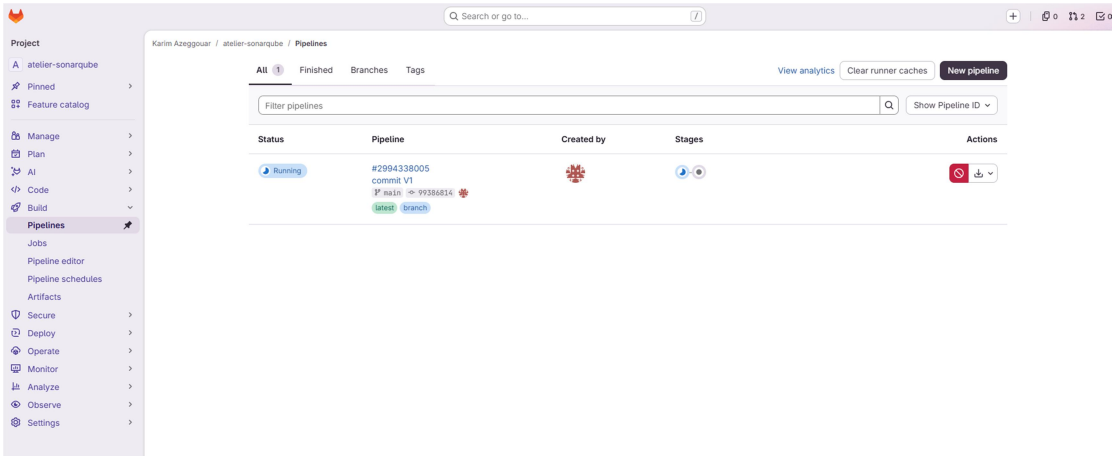
deploy-pages:
  stage: deploy
  image: alpine:latest
  script:
    - mkdir -p public
    - cp index.html style.css script.js public/
    - test -s public/index.html
  pages:
    publish: public
  rules:
    - if: '$CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH'
```

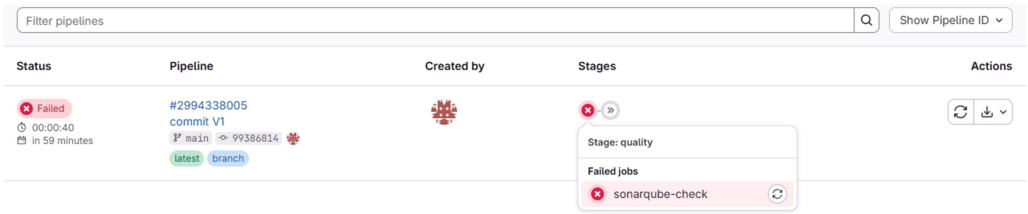
— Envoyer la V1 correcte

Dans le terminal :

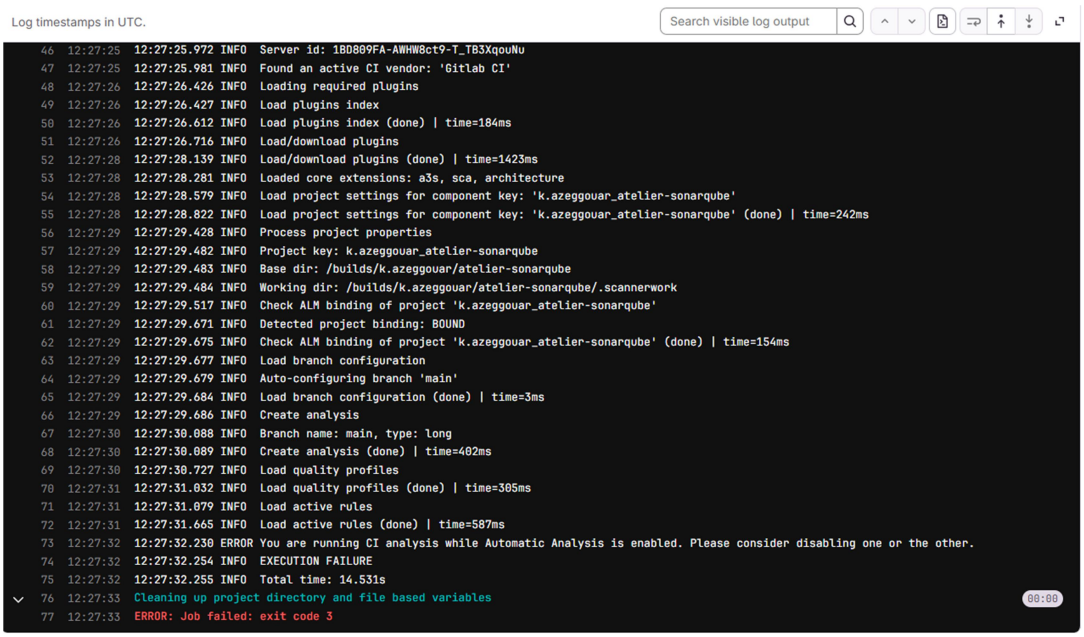
```
git add .
git commit -m "V1 - code correct"
git push
```

— Voir l’état du pipline sur GitLab





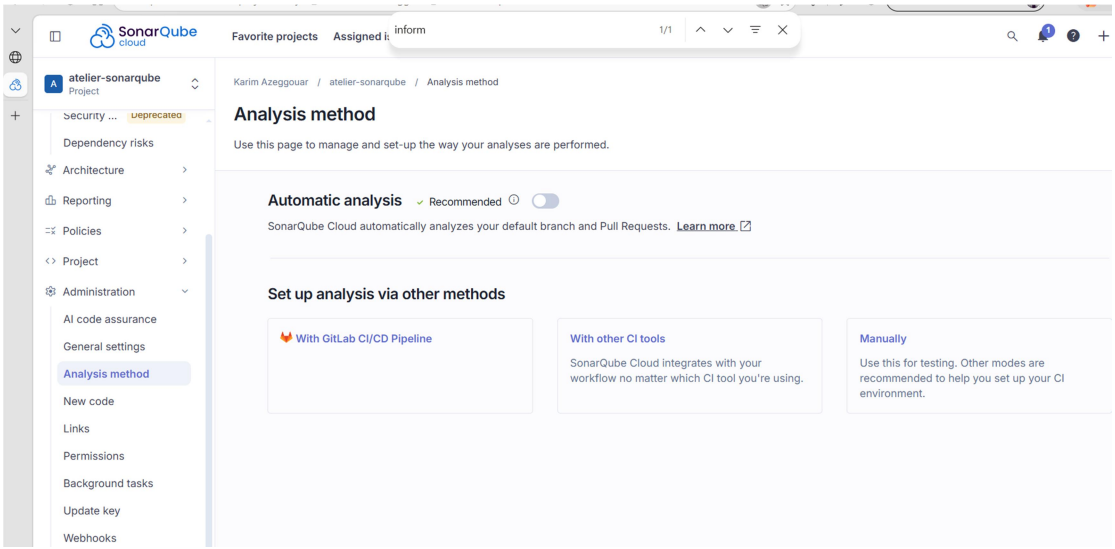
Si le job quality failed :

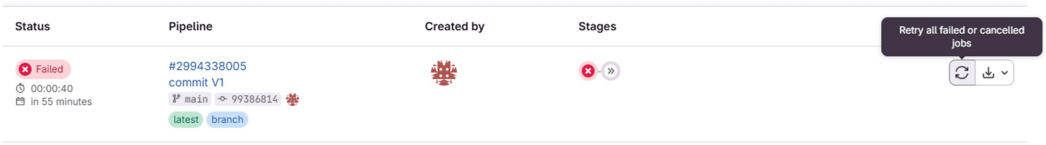


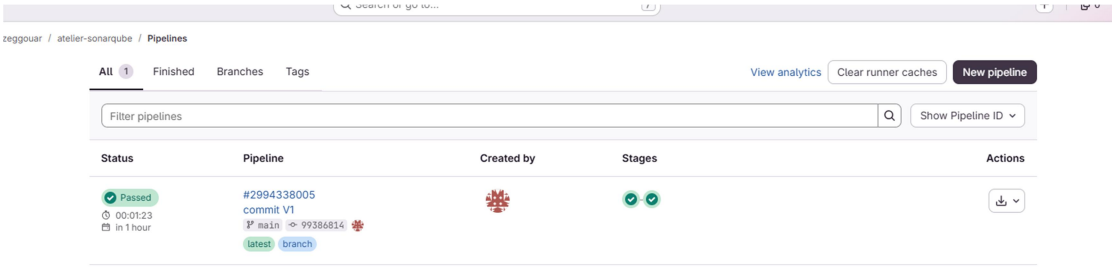
ERROR You are running CI analysis while Automatic Analysis is enabled.
Please consider disabling one or the other.

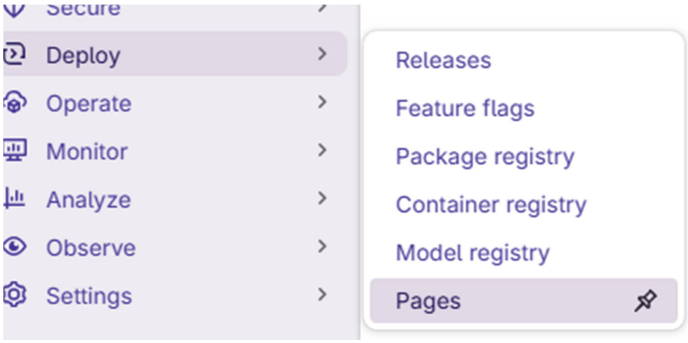
Solution : désactiver Automatic Analysis

1. Retourne sur ton projet `atelier-sonarqube` dans SonarQube Cloud.
2. Dans le menu de gauche, ouvre **Administration → Analysis method**.
3. Repère **Automatic Analysis**.
4. Désactive cette option.
5. Retourne dans GitLab et relance le pipeline avec **Build → Pipelines → Retry**.



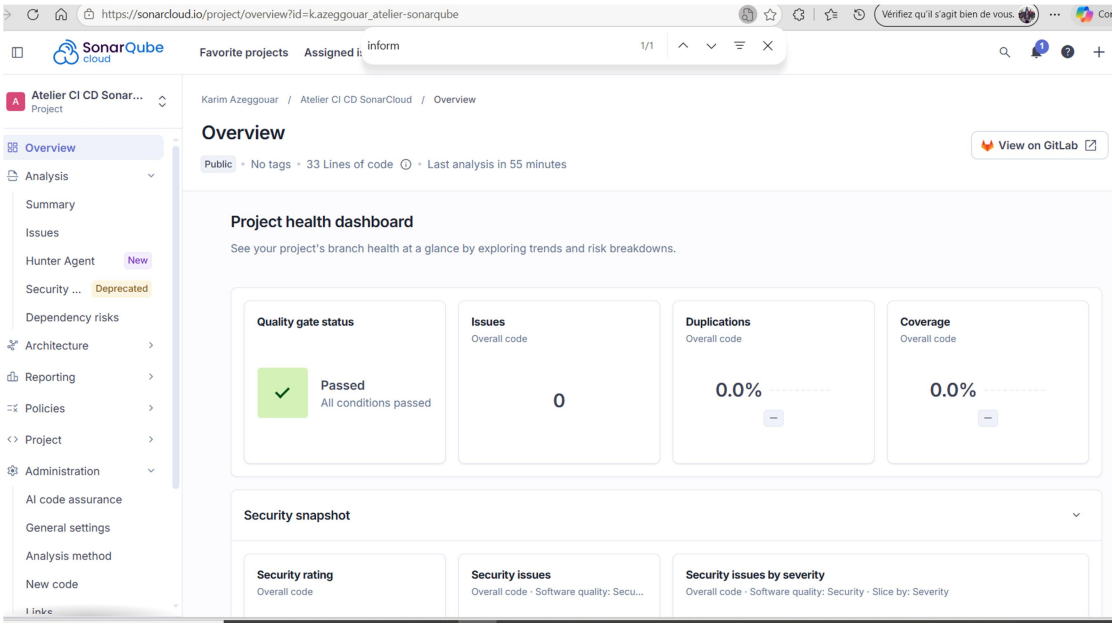

Status
Pipeline
Created by
Stages
Retry all failed or cancelled jobs
Failed
00:00:40
in 55 minutes
#2994338005
commit V1
main
99386814
latest
branch


zeggouar / atelier-sonarqube / Pipelines
All 1
Finished
Branches
Tags
View analytics
Clear runner caches
New pipeline
Filter pipelines
Show Pipeline ID
Status
Pipeline
Created by
Stages
Actions
Passed
00:01:23
in 1 hour
#2994338005
commit V1
main
99386814
latest
branch


Secure
Deploy
Operate
Monitor
Analyze
Observe
Settings
Releases
Feature flags
Package registry
Container registry
Model registry
Pages


atelier-sonarqube-e0f1a7.gitlab.io
(164) Musique pour...
Saisie d'absence IST...
La mélodie du Cora...
Atelier GitLab CI/CD
Version 1 : code correct
Cliquer ici

Dans sonar on remarque : quality gate passed



— Modifier le code pour créer la V2

dans votre V2 : une injection XSS potentielle

```
const form = document.getElementById("demo-form");
const input = document.getElementById("user-input");
const message = document.getElementById("message");

form.addEventListener("submit", (event) => {
  event.preventDefault();

  // Demonstration volontairement vulnérable : une saisie est interprétée comme du
HTML.
  message.innerHTML = input.value;
});
```

```html
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Atelier CI/CD</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Atelier GitLab CI/CD</h1>
  <p>Entrez un texte pour la démonstration :</p>
  <form id="demo-form">
    <label for="user-input">Votre saisie</label>
    <input id="user-input" name="message" type="text" autocomplete="off">
    <button type="submit">Afficher</button>
  </form>
  <p id="message" aria-live="polite"></p>

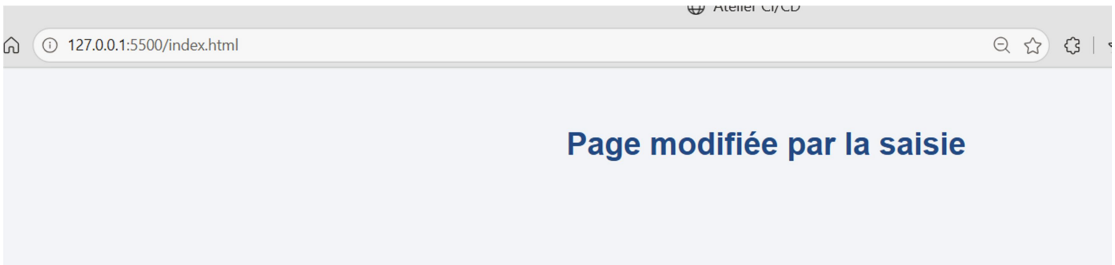
  <script src="script.js"></script>
</body>
</html>
```



Entrer :

```
<img src=x
onerror="document.body.insertAdjacentHTML('beforeend','<h2>Injection
persistante</h2>')">
```

La page est remplacée par le titre **« Page modifiée par la saisie »**,



modifier le .yml :

```yaml
stages:
  - quality
  - security
  - deploy

variables:
  GIT_DEPTH: "0"
  SONAR_USER_HOME: "${CI_PROJECT_DIR}/.sonar"
  SONAR_PROJECT_KEY: "k.azeggouar_atelier-sonarqube"

syntax-check:
  stage: quality
  image: node:22-alpine
  script:
    - node --check script.js
  rules:
    - if: '$CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH'

sonarqube-check:
  stage: quality
  image:
    name: sonarsource/sonar-scanner-cli:latest
    entrypoint: [""]
  script:
    - sonar-scanner -Dsonar.host.url="$SONAR_HOST_URL" -Dsonar.token="$SONAR_TOKEN"
-Dsonar.qualitygate.wait=true
  rules:
    - if: '$CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH'

security-check:
  stage: security
  image: node:22-alpine
  script:
    - |
      node --input-type=module -e '
      const base = process.env.SONAR_HOST_URL;
      const token = process.env.SONAR_TOKEN;
      const project = process.env.SONAR_PROJECT_KEY;
      const branch = process.env.CI_DEFAULT_BRANCH;
      const auth = "Basic " + Buffer.from(`${token}:`).toString("base64");

      async function getSonarJson(url) {
        const response = await fetch(url, { headers: { Authorization: auth } });
        if (!response.ok) {
          throw new Error(`SonarCloud API returned ${response.status}`);
        }
        return response.json();
      }

      const issuesUrl = new URL("/api/issues/search", base);
      issuesUrl.searchParams.set("componentKeys", project);
      issuesUrl.searchParams.set("branch", branch);
      issuesUrl.searchParams.set("types", "VULNERABILITY");
      issuesUrl.searchParams.set("resolved", "false");
      issuesUrl.searchParams.set("ps", "1");
```

```
      const issues = await getSonarJson(issuesUrl);
      const vulnerabilities = issues.paging?.total ?? 0;
      console.log(`Open vulnerabilities: ${vulnerabilities}`);

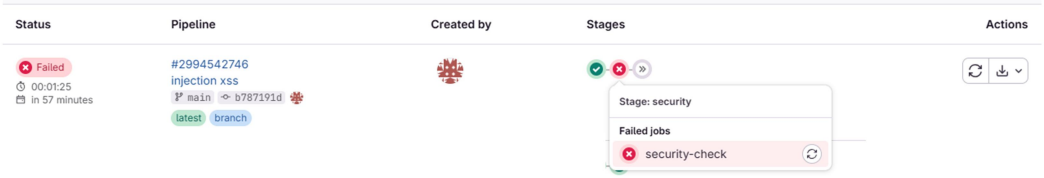
      const hotspotsUrl = new URL("/api/hotspots/search", base);
      hotspotsUrl.searchParams.set("projectKey", project);
      hotspotsUrl.searchParams.set("branch", branch);
      hotspotsUrl.searchParams.set("status", "TO_REVIEW");
      hotspotsUrl.searchParams.set("ps", "1");
      const hotspots = await getSonarJson(hotspotsUrl);
      const hotspotsToReview = hotspots.paging?.total ?? 0;
      console.log(`Hotspots awaiting review: ${hotspotsToReview}`);

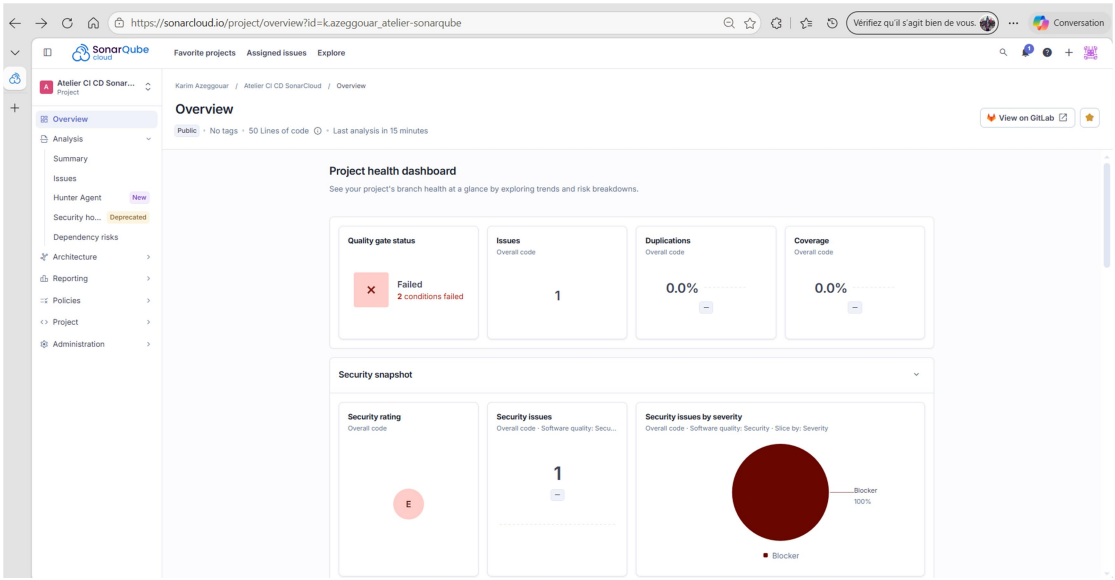
      if (vulnerabilities > 0 || hotspotsToReview > 0) {
        console.error("Failed: SonarCloud found security findings that need
attention.");
        process.exitCode = 1;
      }
      '
  rules:
    - if: '$CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH'

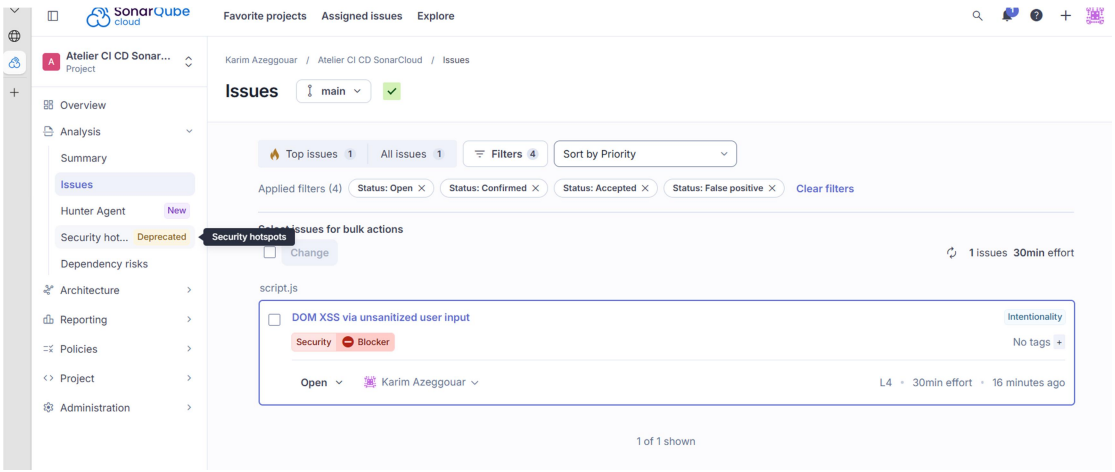
deploy-pages:
  stage: deploy
  image: alpine:latest
  script:
    - mkdir -p public
    - cp index.html style.css script.js public/
    - test -s public/index.html
  pages:
    publish: public
  rules:
    - if: '$CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH'
```

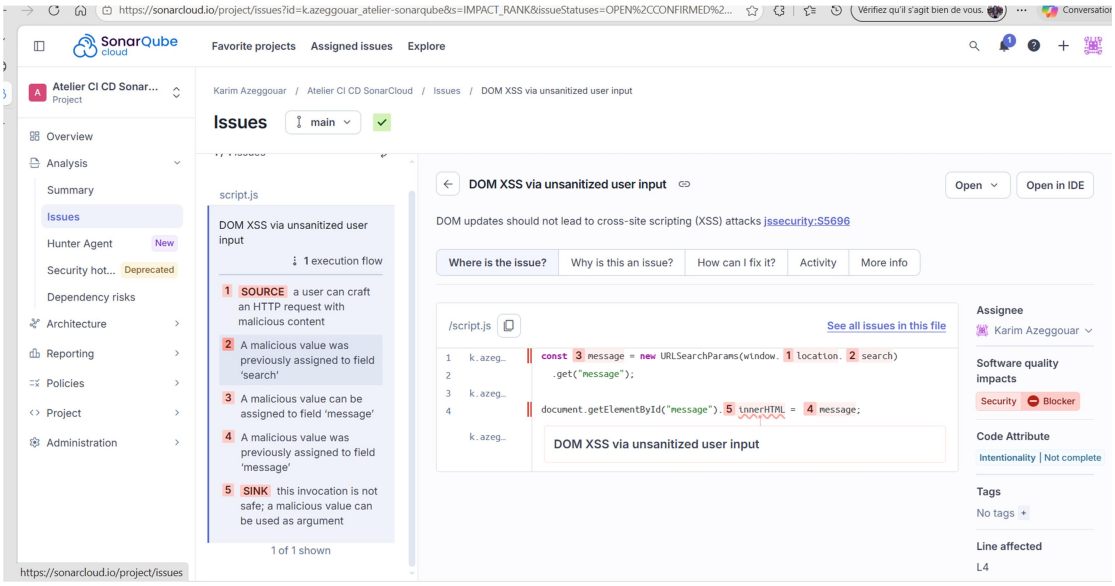
Envoyez la V2 :

```
git add .
git commit -m "V2 - injection xss"
git push
```




https://sonarcloud.io/project/overview?id=k.azeggouar_atelier-sonarqube
Vérifiez qu'il s'agit bien de vous.
Conversation
SonarQube cloud
Favorite projects
Assigned issues
Explore
Atelier CI CD Sonar...
Project
Karim Azeggouar / Atelier CI CD SonarCloud / Overview
Overview
Public • No tags • 50 Lines of code • Last analysis in 15 minutes
View on GitLab
Overview
Analysis
Summary
Issues
Hunter Agent
New
Security ho...
Deprecated
Dependency risks
Architecture
Reporting
Policies
Project
Administration
Project health dashboard
See your project's branch health at a glance by exploring trends and risk breakdowns.
Quality gate status
Failed
2 conditions failed
Issues
Overall code
1
Duplications
Overall code
0.0%
Coverage
Overall code
0.0%
Security snapshot
Security rating
Overall code
E
Security issues
Overall code · Software quality: Secu...
1
Security issues by severity
Overall code · Software quality: Security · Slice by: Severity
Blocker
100%
Blocker


SonarQube cloud
Favorite projects
Assigned issues
Explore
Atelier CI CD Sonar...
Project
Karim Azeggouar / Atelier CI CD SonarCloud / Issues
Issues
main
Overview
Analysis
Summary
Issues
Hunter Agent
New
Security hot...
Deprecated
Security hotspots
Dependency risks
Architecture
Reporting
Policies
Project
Administration
Top issues 1
All issues 1
Filters 4
Sort by Priority
Applied filters (4)
Status: Open
Status: Confirmed
Status: Accepted
Status: False positive
Clear filters
Change
1 issues 30min effort
script.js
DOM XSS via unsanitized user input
Intentionality
Security
Blocker
No tags
Open
Karim Azeggouar
L4 • 30min effort • 16 minutes ago
1 of 1 shown


https://sonarcloud.io/project/issues?id=k.azeggouar_atelier-sonarqube&s=IMPACT_RANK&issueStatuses=OPEN%2CCONFIRMED%2...
SonarQube cloud
Favorite projects
Assigned issues
Explore
Atelier CI CD Sonar...
Project
Karim Azeggouar / Atelier CI CD SonarCloud / Issues / DOM XSS via unsanitized user input
Issues
main
Overview
Analysis
Summary
Issues
Hunter Agent
New
Security hot...
Deprecated
Dependency risks
Architecture
Reporting
Policies
Project
Administration
script.js
DOM XSS via unsanitized user input
1 execution flow
1 SOURCE a user can craft an HTTP request with malicious content
2 A malicious value was previously assigned to field 'search'
3 A malicious value can be assigned to field 'message'
4 A malicious value was previously assigned to field 'message'
5 SINK this invocation is not safe; a malicious value can be used as argument
1 of 1 shown
DOM XSS via unsanitized user input
Open
Open in IDE
DOM updates should not lead to cross-site scripting (XSS) attacks jssecurity:S5696
Where is the issue?
Why is this an issue?
How can I fix it?
Activity
More info
/script.js
See all issues in this file
1 k.azeg... const 3 message = new URLSearchParams(window. 1 location. 2 search)
2 .get("message");
3 k.azeg...
4 document.getElementById("message"). 5 innerHTML = 4 message;
k.azeg...
DOM XSS via unsanitized user input
Assignee
Karim Azeggouar
Software quality impacts
Security
Blocker
Code Attribute
Intentionality | Not complete
Tags
No tags
Line affected
L4
https://sonarcloud.io/project/issues

Conclusion : que se passe-t-il après un `git push` ?

1. Push vers GitLab : le développeur envoie son code.
2. Pipeline CI/CD déclenché : GitLab lance automatiquement les jobs.
3. Analyse SonarQube : le code est analysé pour détecter les bugs, les problèmes de sécurité et de maintenabilité.
4. Quality Gate échoué (`Failed`) : si `sonar.qualitygate.wait=true` et que le job détecte cet échec, le job `quality` échoue.
5. Déploiement bloqué : le job `deploy` ne s'exécute pas, puisque le stage `quality` a échoué.
6. Correction du code : le développeur corrige les problèmes, puis effectue un nouveau `push`. Le pipeline recommence.

À retenir : GitLab CI/CD automatise les étapes, SonarQube contrôle la qualité du code, et le Quality Gate peut empêcher le déploiement d'un code qui ne respecte pas les critères définis.